esame-webapp

Documentazione tecnica del progetto web

Tecnologia principale	Next.js 16 con React e TypeScript
Persistenza	SQLite tramite better-sqlite3
Autenticazione	Server Actions, bcryptjs e cookie httpOnly
Stato del documento	Prima versione - fotografia dello stato attuale del progetto

Documento generato il 28 settembre 2026

Indice

Sezione	Argomento
1	Presentazione del progetto
2	Tecnologie e dipendenze
3	Struttura del progetto
4	Pagine e navigazione
5	Registrazione e autenticazione
6	Database e modello dati
7	Sicurezza
8	Avvio e utilizzo in locale
9	Stato attuale e possibili evoluzioni

Nota di lettura. Questa prima documentazione descrive il codice presente nel repository al momento della redazione. Le funzionalit? non ancora implementate o non collegate alla navigazione vengono indicate come evoluzioni, senza presentarle come gi? disponibili.

1. Presentazione del progetto

esame-webapp ? una piccola applicazione web sviluppata con Next.js. Il progetto rappresenta la base di un negozio online e include un primo sistema di gestione degli utenti: una persona pu? creare un account, autenticarsi e accedere a un?area privata.

L?applicazione ? organizzata secondo l?App Router di Next.js. Le pagine sono definite dalla struttura delle cartelle dentro **app/**, mentre le funzionalit? riutilizzabili e il codice di supporto sono raccolti rispettivamente in **components/** e **lib/**.

Obiettivi attuali

Obiettivo	Descrizione
Interfaccia web	Fornire una home page e una barra di navigazione coerenti con un negozio online.
Gestione utenti	Permettere la registrazione con username e password.
Accesso autenticato	Verificare le credenziali e creare una sessione persistente per l?utente.
Area riservata	Proteggere una pagina accessibile solo dopo il login.
Persistenza locale	Salvare gli utenti in un database SQLite locale al progetto.

2. Tecnologie e dipendenze

Il progetto utilizza un insieme di tecnologie JavaScript/TypeScript moderne, con una separazione tra rendering dell?interfaccia, logica lato server e persistenza dei dati.

Tecnologia	Ruolo nel progetto
Next.js 16.3.5	Framework applicativo: routing, rendering e Server Actions.
React 19.2.8	Libreria per la costruzione dei componenti dell?interfaccia.
TypeScript 5	Tipizzazione statica del codice applicativo.
Tailwind CSS 4	Classi utility utilizzate per layout, colori, spaziature e tema chiaro/scuro.
better-sqlite3	Accesso sincrono al database SQLite.
bcryptjs	Hashing e verifica delle password.

Dipendenze di sviluppo

ESLint e **eslint-config-next** sono configurati per controllare la qualit? del codice. Il file **package-lock.json** blocca le versioni effettivamente installate.

3. Struttura del progetto

La struttura principale del repository ? la seguente:

```
app/
  page.tsx                    Home page
  layout.tsx                  Layout globale e Navbar
  globals.css                 Stili globali e Tailwind
```

```
  login/
    page.tsx                  Form di accesso
    actions.ts                Server Action del login
  registrati/
    page.tsx                  Form di registrazione
    actions.ts                Server Action della registrazione
  area-privata/
    page.tsx                  Pagina protetta
components/
  Navbar.tsx                  Navigazione principale
lib/
  db.ts                       Connessione e inizializzazione SQLite
  session.ts                  Creazione e lettura della sessione
data/
  app.sqlite                  Database locale generato all?avvio
public/                       Risorse statiche
```

Layout globale

app/layout.tsx definisce il layout condiviso da tutte le pagine. Importa gli stili globali, configura i font Geist, imposta i metadati e visualizza **Navbar** sopra il contenuto della pagina.

La lingua del documento HTML ? attualmente impostata su **en**; le etichette dell?interfaccia sono invece principalmente in italiano.

4. Pagine e navigazione

Percorso	Funzione	Accesso
/	Presentazione del negozio online e descrizione sintetica di Next.js.	Pubblico
/registrati	Form per creare un nuovo account.	Pubblico
/login	Form per inserire username e password.	Pubblico
/area-privata	Contenuto disponibile dopo l?autenticazione.	Protetto

La barra di navigazione ? definita in **components/Navbar.tsx**. Da qui l?utente pu? raggiungere Home, registrazione, login e la sezione degli utenti registrati. La rotta **/utenti** ? indicata nella navigazione, ma nel codice attuale non ? presente una pagina corrispondente: ? quindi una funzionalit? da completare.

5. Registrazione e autenticazione

Le operazioni di registrazione e login sono implementate come Server Actions. Il browser invia il form al server, evitando di esporre al client la logica di accesso al database o di hashing.

5.1 Registrazione

La funzione **registraUtente** in **app/registrati/actions.ts** esegue i seguenti passaggi:

Passo	Operazione
1	Legge username e password dal FormData ricevuto.
2	Controlla che entrambi i campi siano valorizzati.
3	Verifica che la password contenga almeno 6 caratteri.

Passo	Operazione
4	Cerca nel database un username gi? esistente.
5	Calcola l?hash della password con bcrypt usando un costo pari a 10.
6	Inserisce il nuovo utente nella tabella Utente.
7	Reindirizza l?utente alla pagina di login.

5.2 Login

La funzione **loginUtente** in **app/login/actions.ts** cerca l?utente tramite username, confronta la password inserita con l?hash salvato usando **bcrypt.compare** e, in caso di successo, invoca **creaSessione**. Infine esegue il redirect verso **/area-privata**. Username inesistente e password errata producono lo stesso messaggio ?Credenziali non valide?, evitando di distinguere quale dato sia sbagliato.

5.3 Protezione dell?area privata

All?inizio del rendering di **app/area-privata/page.tsx** viene chiamata **getUserIdDaSessione**. Se non viene trovato un identificativo valido, la pagina esegue un redirect verso **/login**; in questo modo il controllo viene effettuato lato server prima di mostrare il contenuto.

6. Database e modello dati

Il modulo **lib/db.ts** crea la cartella **data/** se non esiste e apre il file **data/app.sqlite**. All?avvio viene eseguito uno schema idempotente con **CREATE TABLE IF NOT EXISTS**.

```
CREATE TABLE IF NOT EXISTS Utente (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  username TEXT NOT NULL UNIQUE,
  password_hash TEXT NOT NULL,
  created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP
)
```

Il campo **username** ? univoco e impedisce la registrazione duplicata. Il campo **password_hash** contiene solo il risultato dell?hashing, non la password in chiaro. Il pragma **journal_mode = WAL** abilita il Write-Ahead Logging di SQLite, utile per la gestione concorrente delle letture e scritture.

7. Sicurezza

Le misure gi? presenti nel progetto sono:

Misura	Implementazione
Password non in chiaro	bcryptjs genera un hash prima del salvataggio.
Query parametrizzate	Le query usano il placeholder **?** e passano i valori separatamente.
Cookie non leggibile dal client	La sessione viene salvata con l?attributo **httpOnly**.
Cookie sicuro in produzione	L?attributo **secure** ? attivato quando NODE_ENV ? production.
Limitazione CSRF di base	Il cookie usa **sameSite: lax**.
Scadenza sessione	Il cookie ha una durata massima di sette giorni.

La sessione attuale contiene l?ID numerico dell?utente nel cookie. Per un ambiente di produzione sarebbe opportuno valutare una sessione server-side o un token casuale non direttamente correlato all?ID del database, oltre a messaggi d?errore visualizzati nell?interfaccia senza interrompere il rendering con un errore non gestito.

8. Avvio e utilizzo in locale

Prerequisiti: Node.js installato e una shell disponibile nella cartella del progetto.

Installazione delle dipendenze

```
npm install
```

Avvio in modalit? sviluppo

```
npm run dev
```

Dopo l?avvio, l?applicazione ? disponibile normalmente all?indirizzo **http://localhost:3000**. Il database viene inizializzato automaticamente al primo caricamento del modulo **lib/db.ts**.

Comandi disponibili

Comando	Scopo
npm run dev	Avvia il server di sviluppo con aggiornamento automatico.
npm run lint	Esegue ESLint sul progetto.
npm run build	Crea la build di produzione.
npm run start	Avvia l?applicazione dopo una build.

Percorso consigliato di prova

1. Aprire **/registrati** e creare un account con password di almeno sei caratteri.
2. Verificare il redirect a **/login**.
3. Effettuare il login con le credenziali appena create.
4. Verificare l?accesso a **/area-privata**.
5. Provare ad aprire direttamente l?area privata dopo aver rimosso il cookie di sessione: si deve essere reindirizzati al login.

9. Stato attuale e possibili evoluzioni

Il progetto costituisce una base funzionante per la gestione degli utenti, ma la parte di negozio online ? ancora introduttiva. Le evoluzioni pi? naturali sono:

Priorit?	Evoluzione proposta
Alta	Aggiungere logout collegato a **distruggiSessione**.
Alta	Creare la pagina **/utenti** oppure rimuovere il relativo link dalla Navbar.
Alta	Uniformare il link del login: la Navbar usa **/Login**, mentre la cartella definisce **/login**.
Media	Aggiungere gestione degli errori nei form con messaggi visibili all?utente.
Media	Aggiungere validazione pi? completa di username e password.

Priorit?	Evoluzione proposta
Media	Implementare prodotti, catalogo, carrello e ordini.
Bassa	Aggiornare metadati, lingua HTML e descrizione SEO del sito.

Conclusione

La base architetturale ? adatta a estendere il progetto senza spostare la logica sensibile nel client: le Server Actions centralizzano le operazioni di autenticazione, **lib/db.ts** incapsula l?accesso ai dati e **lib/session.ts** raccoglie la gestione della sessione. Le prossime funzionalit? possono quindi essere aggiunte mantenendo questa separazione.